Category C Assignment Problems

EASY TIER

PROBLEM 1 • **Easy**

Fantasy Team Score Multiplier

Topics Integrated: *Creating/Modifying Arrays, Arrays Passed by Reference*

Problem Statement

In a fantasy sports app, every user picks a Captain (2× points) and a Vice-Captain (1.5× points) from their lineup. Write a method that applies both multipliers directly to the lineup's score array, so the app's scoreboard reflects the boosted totals immediately after the match end.

Requirements

- applyMultipliers(...) must modify the caller's original array directly — it should return nothing at all.
- Only the captain's and vice-captain's positions change; every other score stays exactly as it was.

Function Signature(s)

```
static void applyMultipliers(double[] playerScores, int captainIndex, int viceCaptainIndex)
```

Examples

Input	Output

`double[] scores = {40, 55, 30, 62};`
`applyMultipliers(scores, 1, 3);`
`Arrays.toString(scores)`
	`"[40.0, 110.0, 30.0, 93.0]"`

Explanation: *Index 1 (55) is the captain, doubled to 110.0. Index 3 (62) is the vice-captain, multiplied by 1.5 to 93.0. Indices 0 and 2 are untouched.*

HINT This only ever changes two specific positions — you don't need a loop over the whole array at all, just two direct index assignments.

CONSTRAINTS playerScores.length up to 15. captainIndex and viceCaptainIndex are always valid, distinct indices.

PROBLEM 2 • Easy

Duplicate Player Pick Checker

Topics Integrated: *Arrays, Strings, Nested Loops*

Problem Statement

A glitch in the fantasy app's draft screen occasionally lets a user tap the same player twice before the UI catches up, silently adding them to the lineup twice. Write a method that checks a submitted lineup for a repeated player name before it's accepted.

Requirements

- Compare every name against every other name using plain nested loops — no Collections class of any kind.
- Report the first duplicate found, scanning in order; if none exist, say so clearly.

Function Signature(s)

```
static String findDuplicatePick(String[] playerNames)
```

Examples

Input	Output
{"Kohli", "Bumrah", "Kohli", "Rohit"}	"Duplicate Found: Kohli"

Input	Output
{"Kohli", "Bumrah", "Rohit"}	"No Duplicates Found"

HINT For each name, only compare it against the names that come after it in the lineup — you never need to check a pair twice.

CONSTRAINTS Lineup size up to 11 players. Comparison is case-sensitive.

Key Concepts Tested: *Nested-loop pairwise comparison, string equality, early-exit scanning.*

INTERMEDIATE TIER

Problems that need a genuine algorithmic idea, not just array syntax — single-pass tracking, and reusable helper methods on a grid.

PROBLEM 3 • Intermediate

Top Performer Tracker

Topics Integrated: *Arrays, Loops, Logical Thinking*

Problem Statement

A fantasy league's weekly recap wants to instantly call out the week's standout performer and the week's biggest disappointment, along with how wide the gap between them was — without sorting the entire scoreboard just to read off two numbers.

Requirements

- Do not sort the array — find the minimum and maximum in a single pass through the array.
- Report the minimum, the maximum, and the spread (max − min) together.

Function Signature(s)

```
static String findMinMaxSpread(int[] scores)
```

Examples

Input	Output
{45, 82, 79, 90, 33, 90, 61}	"Min: 33 | Max: 90 | Spread: 57"

HINT You only need two running variables, min and max — update both from the same single loop, comparing each score against both at once.

CONSTRAINTS scores.length is at least 2, up to 10,000.

Key Concepts Tested: *Single-pass min/max tracking, avoiding an unnecessary sort.*

PROBLEM 4 • Intermediate

Match Day Grid Analyzer

Topics Integrated: *2D Arrays, User-Defined Methods (reused), Loops*

Problem Statement

A cricket stats app logs runs scored in every over of every match as a grid — one row per match, one column per over. The app wants to flag which matches were genuine "Power Surge" innings (a high scoring rate throughout) without repeating the same averaging code once per match.

Requirements

- Write a private helper, rowAverage(int[] row), and call it once per match from the main method — do not recompute an average inline more than once.
- A match averaging at or above the threshold is a "Power Surge"; below it, "Normal".

Function Signature(s)

```
static double rowAverage(int[] row)
static String classifyMatches(int[][] runsPerOver, int threshold)
```

Examples

Input	Output
{ {4, 6, 8}, {10, 12, 14}, {2, 3, 1} }, threshold = 8	"Match 0: Normal | Match 1: Power Surge | Match 2: Normal"

HINT The outer loop should be the only place that decides Power Surge vs. Normal — rowAverage() should do nothing except compute one match's average and return it.

CONSTRAINTS Matches may have a different number of overs recorded — do not assume every row has the same length.

ADVANCED TIER

A full workflow: overloaded rules, encapsulated objects, and the standard library's own sort working together.

PROBLEM 5 • Advanced

Fantasy League Auto-Draft Ranking Engine

Topics Integrated: *Arrays, Method Overloading, Static Methods, Standard Library (Arrays.sort), Constructors & Encapsulation*

Problem Statement

An auto-draft feature needs to decide which players are draftable and rank them by fantasy points — but the draft rule isn't one simple cutoff. A player with a long track record qualifies on experience alone, but a newer player still needs to be both reasonably experienced and currently fit to make the cut.

Requirements

- Provide two overloaded isDraftable(...) checks: a matches-played-only rule for established players, and a combined matches-and-fitness rule for everyone else — decide the exact thresholds yourself.
- Player must implement Comparable<Player> so that Arrays.sort(...) alone can rank the draftable array by fantasy points, descending, with no separate comparison logic written elsewhere.

Function Signature(s)

```
public Player(String name, int matchesPlayed, double battingAverage, boolean injured)
static boolean isDraftable(int matchesPlayed)
static boolean isDraftable(int matchesPlayed, boolean injured)
int compareTo(Player other)
static String draftAndRank(Player[] players)
```

Examples

Input	Output

`draftAndRank({`
`  new Player("Virat", 15, 48.0, false),`
`  new Player("Rahul", 7, 55.0, false),`
`  new Player("Sameer", 3, 60.0, false),`
`  new Player("Dev", 12, 20.0, true)`
`})`
	`"1. Rahul | 2. Virat | 3. Dev"`

Explanation: *Virat and Dev both clear the experience-only bar (matches ≥ 10) regardless of fitness — Dev makes the cut despite being injured. Rahul falls short on experience alone, but clears the combined fitness-and-matches check. Sameer clears neither rule and is left out, despite having the best batting average of anyone.*

HINT Once every draftable Player correctly implements compareTo(...), a plain Arrays.sort(draftableArray) does the entire ranking for you — resist the urge to write your own sort loop here.

CONSTRAINTS matchesPlayed is a non-negative integer. Up to 500 players in a batch.